Object Detection on ARM + FPGA using PYNQ-Z2

Repository Name: Object-Detection-ARM-FPGA-PYNQ

1. Abstract

This project demonstrates a heterogeneous edge-AI system combining an ARM processor and FPGA accelerator using the PYNQ-Z2 development board. The objective is to accelerate real-time object detection by offloading computationally intensive operations to hardware while keeping neural network inference on the ARM processor.

The FPGA performs parallel image preprocessing, while the ARM processor runs a Python-based object detection model. This architecture reduces latency and CPU load, proving the effectiveness of hardware-software co-design for embedded AI applications.

2. Introduction

Edge AI systems require:

- Low latency
- Low power consumption
- Real-time processing
- Minimal cloud dependency

Traditional CPU-only systems struggle to meet these constraints.
FPGAs solve this by providing **massively parallel computation**.

This project integrates:

Component	Role
ARM Cortex-A9	Neural network inference
FPGA Fabric	Hardware acceleration
Python	Control & model execution
AXI DMA	Data transfer
CNN Accelerator (HLS)	Parallel convolution

3. System Architecture

Processing Flow

Camera/Image

↓

Python (ARM)

↓

AXI DMA

↓

FPGA CNN Accelerator

↓

Processed Image

↓

Object Detection Model

↓

Bounding Box Output

4. Hardware Platform

Board: PYNQ-Z2
SoC: Zynq-7020

Resource	Usage
ARM PS	Model inference
PL (FPGA)	Convolution acceleration
AXI Lite	Control signals
AXI MM	Memory transfer
DDR Memory	Image buffers

5. Hardware Accelerator (HLS Design)

CNN Convolution Accelerator

The accelerator performs:

- 3×3 convolution
- Sliding window processing
- ReLU activation

HLS Implementation

```
void cnn_accel(data_t input[32][32],
               data_t kernel[3][3],
               data_t output[30][30]) {
#pragma HLS INTERFACE m_axi port=input bundle=gmem
#pragma HLS INTERFACE m_axi port=kernel bundle=gmem
#pragma HLS INTERFACE m_axi port=output bundle=gmem
#pragma HLS INTERFACE s_axilite port=return
for(int i=0;i<30;i++){
 for(int j=0;j<30;j++){
  data_t sum=0;
  for(int ki=0;ki<3;ki++)
   for(int kj=0;kj<3;kj++)
    sum+=input[i+ki][j+kj]*kernel[ki][kj];

  if(sum<0) sum=0;
  output[i][j]=sum;
 }
}
}
```

6. Software Implementation (Python Control)

Load Overlay

```
from pynq import Overlay

ol = Overlay("cnn_design_wrapper.bit")
```

Run Accelerator

```
from pynq import allocate

import numpy as np


inp = allocate((32,32),dtype=np.float32)

ker = allocate((3,3),dtype=np.float32)

out = allocate((30,30),dtype=np.float32)


ol.cnn_accel_0.write(0x10, inp.physical_address)

ol.cnn_accel_0.write(0x18, ker.physical_address)

ol.cnn_accel_0.write(0x20, out.physical_address)

ol.cnn_accel_0.write(0x00, 1)
```

7. Object Detection Model

The ARM processor runs a lightweight Python object detection model.

Steps:

1. Capture frame
2. Send to FPGA preprocessing
3. Run detection
4. Display bounding boxes

8. Results

Metric	Result
Execution	Successful
Detection	Real-time
CPU Usage	Reduced
Acceleration	Hardware verified

9. Hardware Utilization

Resource	Utilization
LUTs	Low
BRAM	Medium
DSP	Used for convolution
Clock	100 MHz

10. Optimization Techniques

- Hardware convolution acceleration
- AXI DMA burst transfer
- Parallel sliding window processing
- ReLU in hardware
- CPU offloading

11. Applications

- Smart surveillance
- Traffic monitoring
- Edge robotics
- Autonomous vehicles
- Smart cities

12. Conclusion

This project successfully demonstrates a heterogeneous computing system combining FPGA acceleration and ARM-based neural network inference. The architecture significantly reduces processing load while maintaining real-time performance, making it suitable for edge AI applications.

13. Future Work

- Full CNN on FPGA
- Quantized neural networks
- Multi-object detection
- Hardware YOLO accelerator